0033. Search in Rotated Sorted Array

Category: Binary Search | LeetCode: https://leetcode.com/problems/search-in-ro

Problem

- Search target in a rotated sorted array of distinct values; return the index or -1 in O(log n).
- Example: [4,5,6,7,0,1,2], target 0 → 4.

Intuition

- At any mid, at least one half ([lo..mid] or [mid..hi]) is sorted.
- Check whether target lies within the sorted half's range; if yes go there, otherwise go to the other half.

Brute Force

- Linear scan: O(n). Or find the pivot (problem 153) then binary search one side.

Optimal Approach

- If nums[mid] == target, return mid.
- If nums[lo] <= nums[mid] (left sorted): target in [nums[lo], nums[mid]) ? hi = mid-1 : lo = mid+1.
- Else (right sorted): target in (nums[mid], nums[hi]] ? lo = mid+1 : hi = mid-1.

Worked Example

```
[4,5,6,7,0,1,2], target 0.

lo=0 hi=6 mid=3 (7): left [4..7] sorted, 0 not in it -> lo=4
lo=4 hi=6 mid=5 (1): left [0..1] sorted, 0 in it    -> hi=4
lo=4 hi=4 mid=4 (0): found -> 4
```

Java Solution

```java
public class SearchInRotatedSortedArray {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[lo] <= nums[mid]) {
                if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
                else lo = mid + 1;
            } else {
                if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
                else hi = mid - 1;
            }
        }
        return -1;
    }
}
```

Complexity

- Time: O(log n). Space: O(1).

Pitfalls

- Use <= in nums[lo] <= nums[mid] so lo == mid counts as a sorted left half.